The Compositional Depth Decay is a Dispatch Tax

Parse-Given Recursive Evaluation Collapses the Exponential Decay Rate

Juan Cruz Dovzak
Independent Researcher
juandovzak@gmail.com

April 2026 — Technical Report v0.1

Version 0.1. This is a strategic priority record; the full technical report with implementation details, training recipes, and the complete benchmark suite is under preparation. Please cite the DOI issued with this release.

Abstract

We study out-of-distribution compositional depth extrapolation on modular reverse-Polish arithmetic and Boolean formula evaluation, and propose that compositional depth decay in neural architectures is not an inherent property of the task but a measurable *dispatch tax* — the per-depth cost of learning *when* to combine intermediate states during forward pass. Concretely, we observe empirically that across a wide range of architectures (Transformer, multi-head attention with typed operators, differentiable soft stacks), out-of-distribution accuracy at test depth d follows a two-parameter law

$$\mathrm{acc}(d) = \mathrm{floor}_{\mathrm{arch}} + (\mathrm{acc}(d_{\min}) - \mathrm{floor}_{\mathrm{arch}}) \exp(-\alpha_{\mathrm{arch}}(d - d_{\min})),$$

where α_{arch} fingerprints the architecture. Leave-one-out prediction of unseen depths from fit on in-training depths achieves mean absolute error of 1.7–3.2 percentage points across 42 runs. We then introduce an architectural intervention that collapses α to zero by removing the need to learn dispatch: a recursive shared-cell evaluator that receives the expression parse tree as a given. On balanced reverse-Polish modular arithmetic at $p = 7$, the intervention achieves 100% accuracy at test depth $d = 8$ (out-of-distribution), and extends to depth 12 at 93–100%. We verify the mechanism directly: the trained cell reproduces the complete $\mathbb{Z}/7\mathbb{Z}$ operator table exactly (147 of 147 triples), transfers bidirectionally between distinct parse-tree shapes without retraining, and its accuracy collapses to chance under parse corruption, establishing a causal dependence on the parse. We do not claim that recursive or tree-augmented neural networks are novel. Our contribution is to (i) quantify compositional depth decay as an architectural property, (ii) identify dispatch discovery as its causal origin, and (iii) demonstrate that parsing, local-operator learning, and recursive evaluation are separable bottlenecks — with tree-recursive shared-cell evaluators addressing the third directly. Full architectural details, training recipes, and the complete benchmark suite are in preparation; this document fixes the priority of the findings.

1 Setup and controls

Tasks. Balanced reverse-Polish notation (RPN) over $\mathbb{Z}/p\mathbb{Z}$ with operators $\{+, -, \times\}$: depth-d expression $[a_0, a_1, \ldots, a_d, \mathrm{op}_1, \ldots, \mathrm{op}_d]$, evaluated by a stack machine with max stack depth $d + 1$. We also evaluate Boolean formulas with operators $\{\textsc{and}, \textsc{or}, \textsc{xor}\}$ in the same layout.

Null model. Modular multiplication collapses on zero; the empirical label distribution is non-uniform. We use rejection sampling to produce label-uniform data. All chance baselines in the paper are the corrected $1/p$, not the majority-class frequency. An earlier internal draft used the uniform baseline without correction, which inflated the reported architectural gap by approximately a factor of two.

Data discipline. Training and test samples are generated by independent seeds. For the smallest depth where the combinatoric space is fully covered by the training budget, train/test overlap is mathematically unavoidable; out-of-distribution claims are at $d \geq 5$, where overlap with training is zero.

Architectures compared. Four families, all near-matched in parameter count for the main comparisons:

- **Transformer** (flat causal attention).
- **MoTE** (attention with a typed operator mixture).
- **Soft stack + typed operators** (differentiable stack, learned push/pop controller).
- **Tree-recursive shared-cell evaluator** (intervention; forward pass traverses parse tree, one learned cell applied at each internal node).

Training. Standard recipe: AdamW, warmup-stable-decay schedule, weight decay 1.0, gradient clip 1.0. We use 15K–50K training steps depending on the experiment. Label distribution is rejection-sampled uniform.

2 The two-parameter empirical law

Across all architectures and curricula studied, the accuracy-vs-depth curve is well-described by Eq. 1 (Abstract). The one-parameter form with floor = chance underfits: admitting floor as a free parameter improves mean R^2 from 0.795 to 0.826 across 28 runs with ≥ 5 depth evaluations.

Table 1: Pooled (floor, α) from three seeds per architecture under multi-depth curriculum.

Architecture	floor	α	Half-life (depth steps)
Transformer	0.16 ± 0.02	0.78 ± 0.18	0.89
MoTE	0.16 ± 0.01	0.76 ± 0.05	0.91
Differentiable stack	0.22 ± 0.02	0.42 ± 0.09	1.66

Fitting on $d \in \{2, 3, 4, 5\}$ and predicting $d \in \{6, 7, 8\}$, mean absolute error is 1.7–3.2pp across architectures.

3 The dispatch tax hypothesis

We propose that α_{arch} measures the per-depth cost of learning *dispatch* — the decision of when to combine intermediate states during the forward pass. Architectures that must infer execution structure from sequential tokens pay this cost cumulatively with depth. The prediction: an architecture whose computational graph already encodes the parse tree should exhibit $\alpha \approx 0$.

Two lines of indirect evidence support the hypothesis before the direct intervention:

1. Probing the learned soft-stack controller reveals it spontaneously assigns high probability to the correct algorithmic actions without supervision (digit tokens trigger push with ≈ 0.91; operator tokens trigger pop-apply with ≈ 0.64).
2. Summed per-layer dispatch entropy correlates monotonically with the fitted α across training curricula: broader training reduces controller uncertainty and, in tandem, the decay rate.

4 Causal intervention: recursive parse-given evaluation

We test the extreme-case prediction by constructing a recursive shared-cell architecture whose forward pass traverses the parse tree and applies a single learned cell at each internal node. Details of the cell and architecture are reserved for the full technical report; the essential property is that dispatch is not learned but structural.

Table 2: $p = 7$ balanced RPN, curriculum $\{2, 3, 4, 5\}$. Chance $= 1/7 \approx 0.143$.

Architecture	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$	$d = 8$
Transformer	0.47	0.27	0.24	0.23	0.19	0.17	0.17
Differentiable stack	0.47	0.37	0.33	0.24	0.20	0.20	0.20
Tree-recursive (ours)	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**	**1.00**

Across three seeds, the intervention's $d = 8$ accuracy is 0.99 ± 0.01. Extrapolating to depths 9–12 (training range was $\{2, 3, 4, 5\}$), accuracy is $0.98, 0.98, 0.94, 0.96$. A 40–80$\times$ reduction in the fitted decay rate vs. flat architectures.

5 Mechanistic verification

The cell learns the operator table exactly. Extracting the trained cell and evaluating it on all (a, b, op) triples at $p = 7$ gives 147 of 147 correct. At $p = 11$ with 25% training coverage of the operator table, the same architectural family achieves $> 99\%$ at $d = 8$, while a simpler MLP cell with recursive sharing only reaches 17% — indicating that typed operators matter for sample-efficient learning, while recursive sharing is what eliminates depth decay structurally.

The cell transfers across parse shapes. We train on one parse convention (balanced RPN, right-skewed trees) and evaluate the same cell, without retraining, on a different parse convention (linear RPN, left-skewed trees). Accuracy is ≥ 0.98 at $d \leq 4$ and remains ≥ 0.79 at $d = 8$ in either direction. The cell is structure-agnostic.

Parse corruption collapses accuracy to chance. Randomizing operand positions at test time with increasing corruption rate c, we observe a monotone degradation from 100% at $c = 0$ to 16% ($\approx$ chance) at $c = 1$ for $d = 8$ OOD. The model's accuracy depends *causally* on the parse, not on a parse-invariant surface pattern.

Table 3: Parse corruption. Training clean; test shuffles operand positions with rate c.

Test depth	$c = 0$	$c = 0.1$	$c = 0.25$	$c = 0.5$	$c = 1.0$
2	1.000	0.934	0.872	0.748	0.528
5	1.000	0.910	0.784	0.594	0.190
8	0.996	0.892	0.808	0.632	0.158

6 Decomposition and boundaries

The result supports a decomposition of compositional generalization into three components:

1. **Parsing** (mapping tokens to a computation graph).
2. **Local operator learning** (fitting the per-node function).

3. **Recursive evaluation** (applying the function along the graph).

Flat architectures must learn all three simultaneously and pay cumulative dispatch tax. The intervention addresses only component (3) by receiving the parse and sharing a cell recursively. Components (1) and (2) remain standard problems: the intervention does not parse, and its cell requires sufficient training coverage of the local operator.

At $p = 31$, the operator table grows to 2883 triples; under our training budget (200K samples per depth, 60K steps), the cell does not fully converge. The intervention nevertheless outperforms Transformer by a factor of three over chance at $d = 5$ OOD (5.1× chance vs. 1.4× chance). This is a cell-learning bottleneck, not an evaluation-structure failure.

Similarly, training with operand values restricted to $\{0, 1, 2, 3\}$ and testing on $\{0, \ldots, 6\}$ shows that value-space extrapolation is not solved by the architecture. Partial interpolation is observed ($\approx 45\%$ on $d = 2$ vs. memorization lower bound of $\approx 30\%$), but not full.

7 What this does not claim

- We do not claim that recursive or tree-augmented neural networks are new. Prior work on Tree-LSTMs, neural module networks, neural programmer-interpreters, neural stack machines, stack-attention Transformers, and tree stack memory units has explored structural inductive biases for compositional learning.
- We do not claim to solve natural-language compositional reasoning. The intervention requires an explicit parse.
- We do not claim that cell learning is trivial. It is a standard machine-learning problem that remains subject to coverage, capacity, and optimization constraints at larger modular arithmetic.
- We do not claim our fit numbers hold outside the tested regime (model size $\leq$ 3.2M, training budget $\leq$ 50K steps, parse-given structured tasks).

8 Corrections to earlier drafts

Early versions of this work reported architectural gaps against a uniform $1/p$ baseline. The empirical label distribution of balanced RPN mod-7 is not uniform (label 0 appears $\approx 20\%$ due to multiplication collapsing on zero). Correcting to rejection-sampled uniform labels reduces the reported gap by approximately a factor of two. All numbers in this document use the corrected baseline.

One single-seed configuration ($d_{\text{model}} = 128$, balanced, 15K steps) showed an anomalous +16pp spike that did not reproduce on extended training. It has been removed as outlier noise.

9 Availability

This document is accompanied by a Zenodo release containing (i) the PDF, (ii) a compressed bitácora of experimental decisions and results, (iii) the claims manifest, (iv) commit hashes of the local implementation, and (v) a README describing how to reproduce the main results. A full technical report including architectural details, training recipes, the benchmark suite, and the implementation is in preparation and will be released separately.

Acknowledgements

We thank the anonymous advisors whose rigorous feedback on intermediate drafts shaped the framing of this work, and the GPT-5.5 research assistant for identifying the null-model calibration error that reduced the earlier reported gaps and strengthened the final claim.

References

[1] Lake, B. M., Baroni, M. (2018). *Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks.* ICML.

[2] Kim, N., Linzen, T. (2020). *COGS: A Compositional Generalization Challenge Based on Semantic Interpretation.* EMNLP.

[3] Petty, J., van Steenkiste, S., Linzen, T. (2025). *The Impact of Depth on Compositional Generalization in Transformer Language Models.* TMLR / arXiv:2310.19956.

[4] Tai, K. S., Socher, R., Manning, C. D. (2015). *Improved Semantic Representations from Tree-Structured Long Short-Term Memory Networks.* ACL.

[5] Reed, S., de Freitas, N. (2016). *Neural Programmer-Interpreters.* ICLR.

[6] DuSell, B., Chiang, D. (2024). *Stack Attention: Improving the Ability of Transformers to Model Hierarchical Patterns.* ICLR.